

6.1.4 链式前向星

链式前向星采用了一种静态链表存储方式，将边集数组和邻接表相结合，可以快速访问一个节点的所有邻接点，在算法竞赛中被广泛应用。

链式前向星有如下两种存储结构。

(1) 边集数组：edge[]，edge[i]表示第 i 条边。

(2) 头节点数组：head[]，head[i]存储以 i 为起点的第 1 条边的下标（edge[]中的下标）

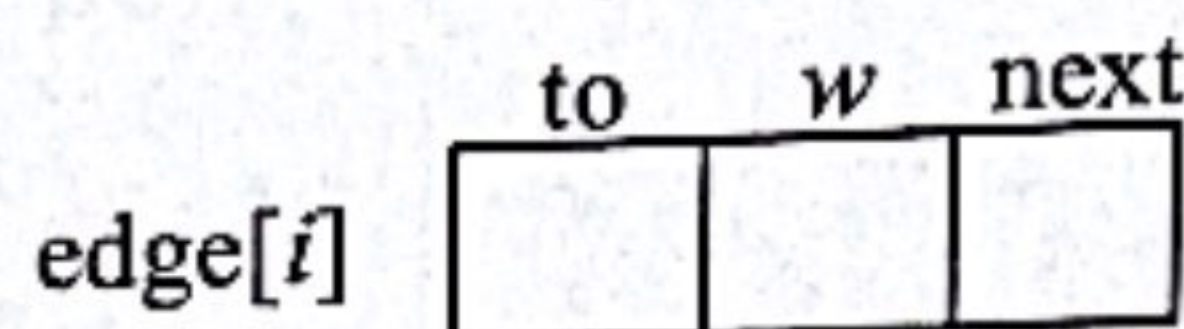
```
struct node{  
    int to,next,w;
```



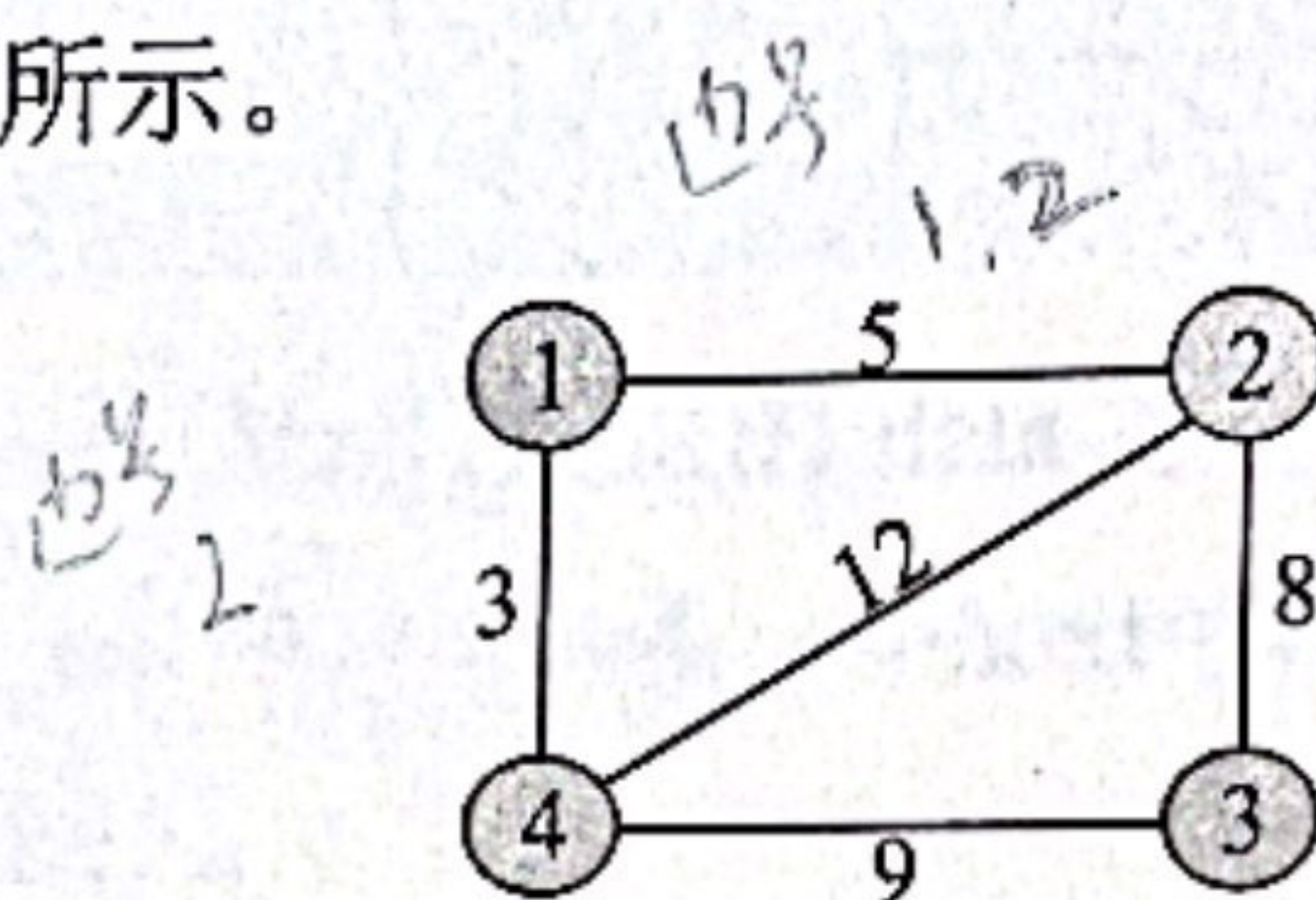
```

}edge[maxe]; //边集数组，对边数一般要设置比 maxn*maxn 大的数，题目有要求除外
int head[maxn]; //头节点数组
    
```

每一条边的结构都如下图所示。

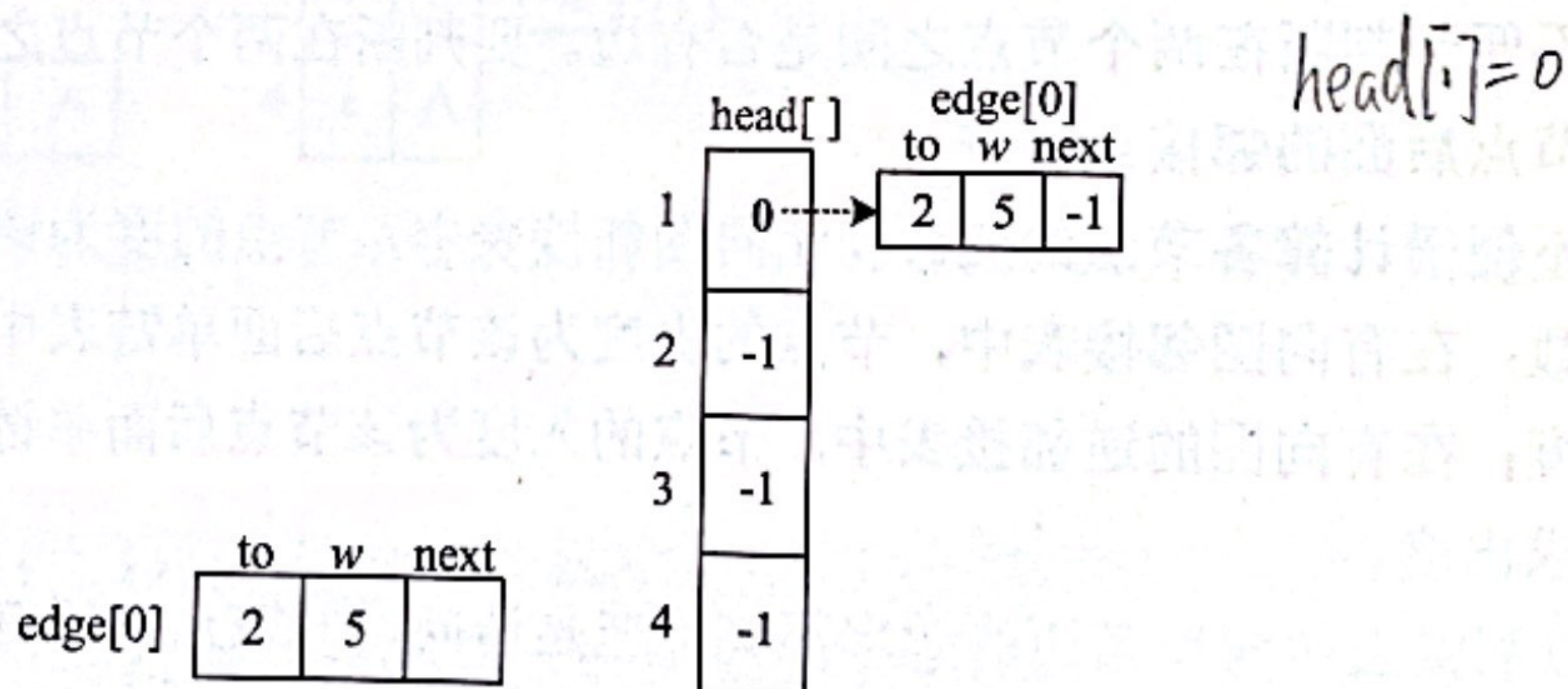


例如，一个无向图如下图所示。

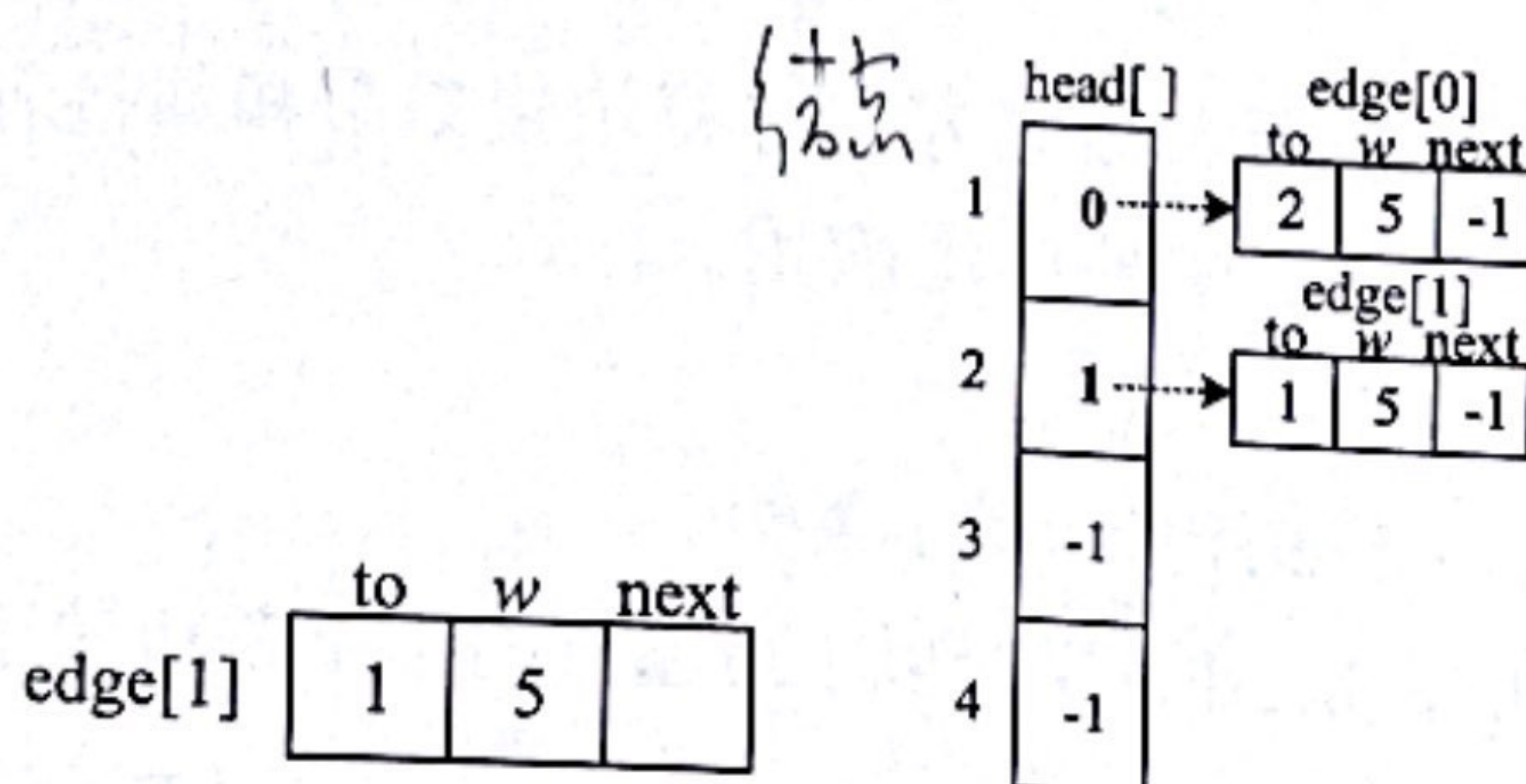


按以下顺序输入每条边的两个端点，建立链式前向星，过程如下。

(1) 输入 1 2 5。创建一条边 1-2，权值为 5，创建第 1 条边 $edge[0]$ ，将该边链接到节点 1 的头节点中（初始时 $head[]$ 数组全部被初始化为 -1）。即 $edge[0].next = head[1]$; $head[1] = 0$ ，表示节点 1 关联的第 1 条边为 0 号边，如下图所示。图中的虚线箭头仅表示它们之间的链接关系，不是指针。

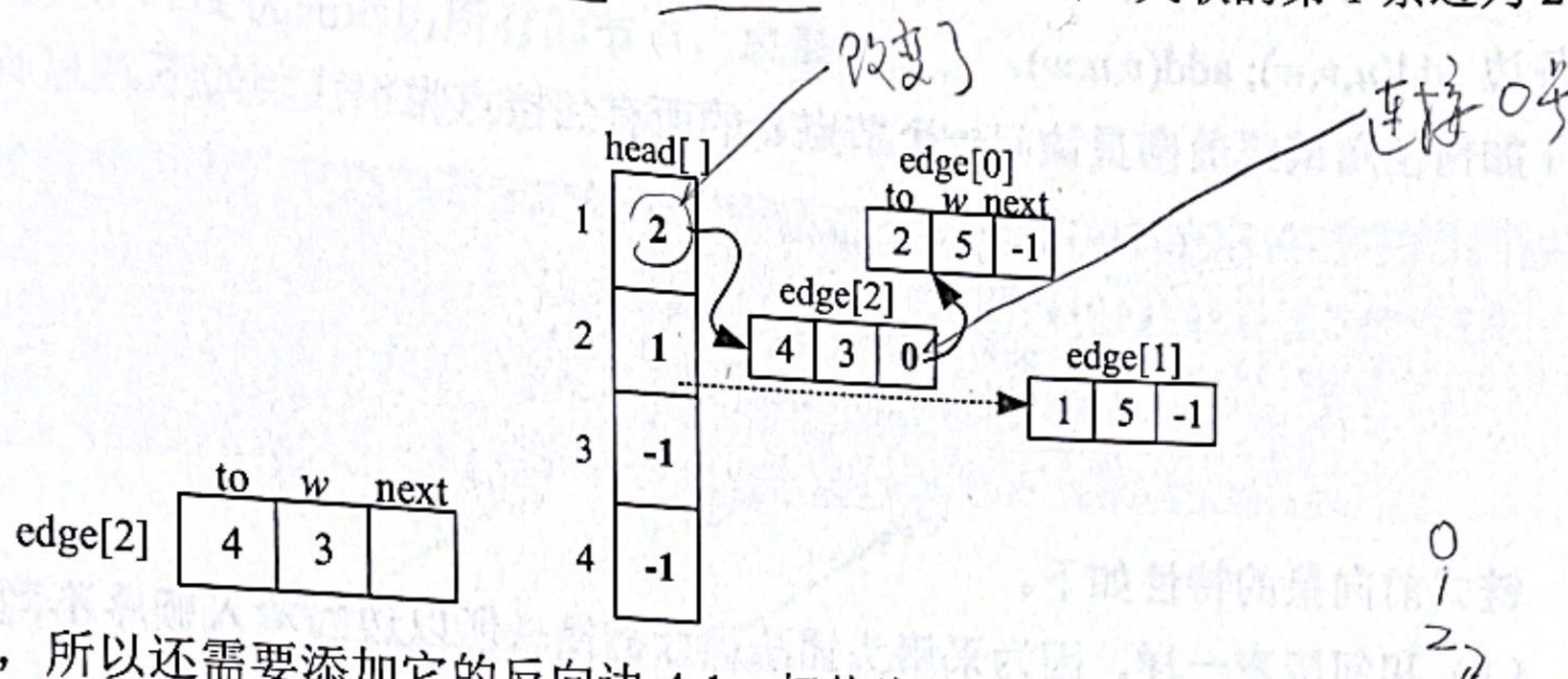


因为是无向图，所以还需添加它的反向边 2-1，权值为 5。创建第 2 条边 $edge[1]$ ，将该边链接到节点 2 的头节点中。即 $edge[1].next = head[2]$; $head[2] = 1$ ；表示节点 2 关联的第 1 条边为 1 号边，如下图所示。

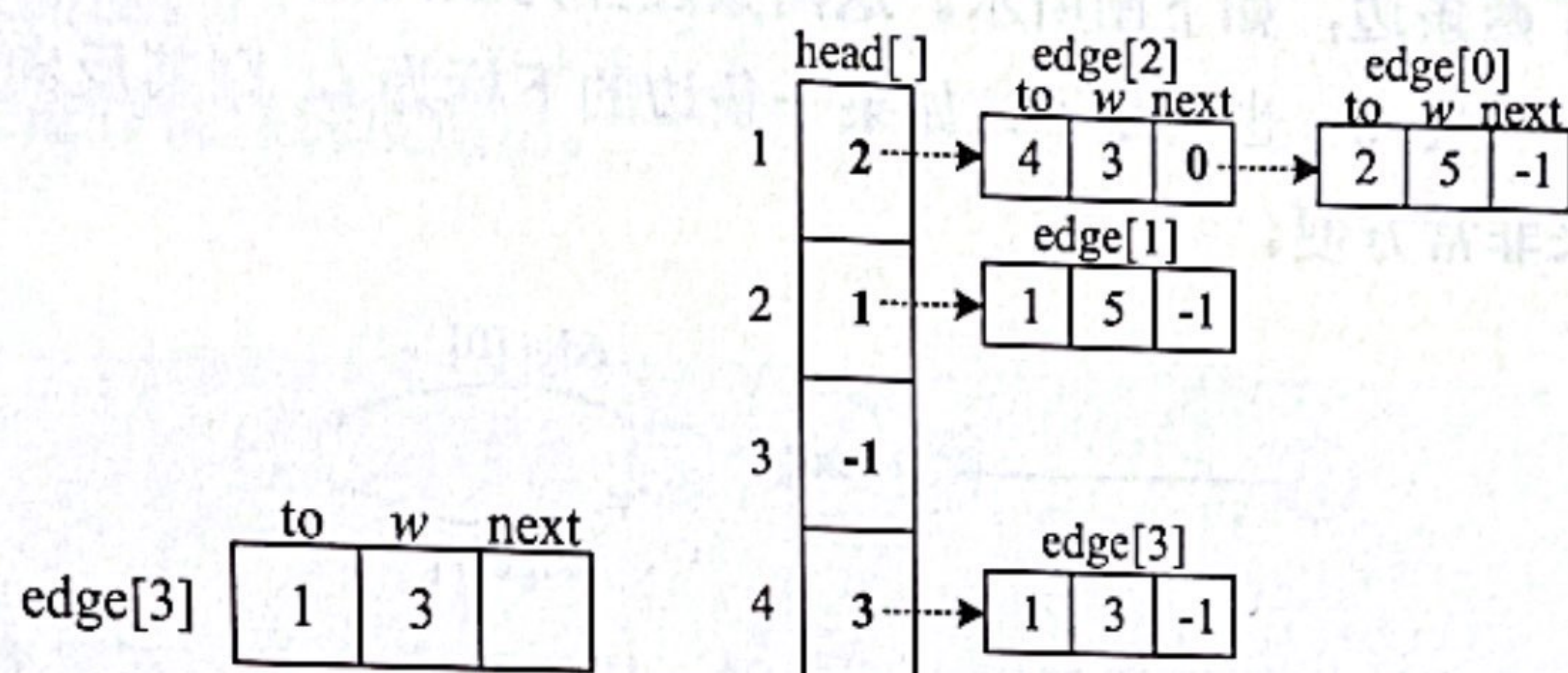


(2) 输入 1 4 3。创建一条边 1-4，权值为 3。创建第 3 条边 $edge[2]$ ，将该边链接到节点 1

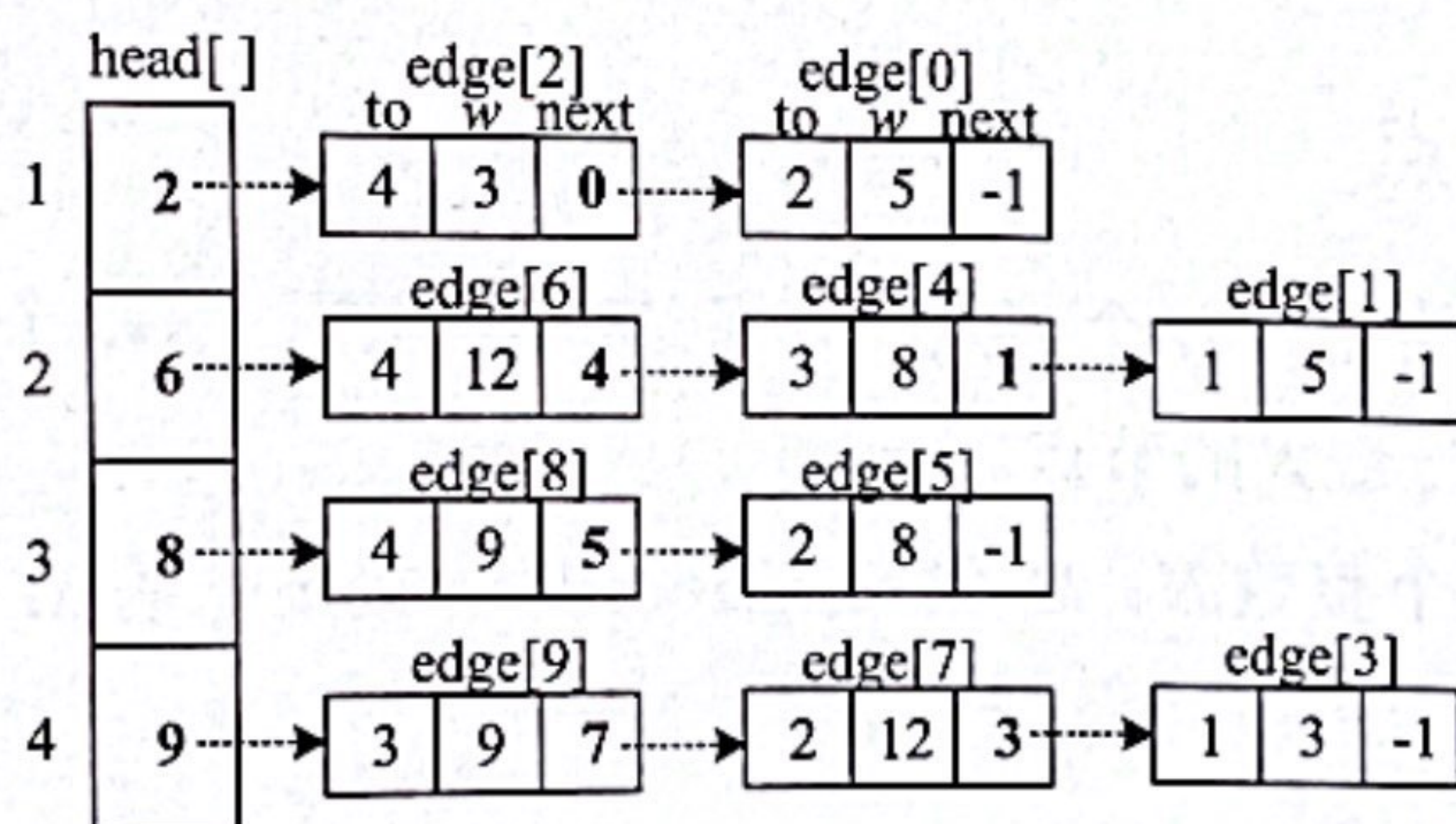
的头节点中（头插法）。即 $\text{edge}[2].\text{next}=\text{head}[1]$; $\text{head}[1]=2$ ，表示节点 1 关联的第 1 条边为 2 号边，如下图所示。



因为是无向图，所以还需要添加它的反向边 4-1，权值为 3。创建第 4 条边 $\text{edge}[3]$ ，将该边链接到节点 4 的头节点中。即 $\text{edge}[3].\text{next}=\text{head}[4]$; $\text{head}[4]=3$ ，表示节点 4 关联的第 1 条边为 3 号边，如下图所示。



(3) 依次输入三条边 2 3 8、2 4 12、3 4 9，创建的链式前向星如下图所示。



添加一条边 $u v w$ 的代码如下：

```
void add(int u, int v, int w) { // 添加一条边
    edge[cnt].to = v;
    edge[cnt].w = w;
    edge[cnt].next = head[u];
    head[u] = cnt++;
}
```


| 算法训练营：海量图解+竞赛刷题（入门篇）

如果是有向图，则每输入一条边，都执行一次 $\text{add}(u, v, w)$ 即可；如果是无向图，则需要添加两条边 $\text{add}(u, v, w); \text{add}(v, u, w)$ 。

如何使用链式前向星访问一个节点 u 的所有邻接点呢？代码如下。

```
for(int i=head[u]; i!=-1; i=edge[i].next) { //i!=-1 可以写为 ~i
    int v=edge[i].to; //u 的邻接点
    int w=edge[i].w; //u-v 的权值
    ...
}
```

链式前向星的特性如下。

(1) 和邻接表一样，因为采用头插法进行链接，所以边的输入顺序不同，创建的链式前向星也不同。

(2) 对于无向图，每输入一条边，都需要添加两条边，互为反向边。例如，输入第 1 条边 1 2 5，实际上添加了两条边，如下图所示。这两条边互为反向边，可以通过与 1 的异或运算得到其反向边， $0^1=1$ ， $1^1=0$ 。也就是说，如果一条边的下标为 i ，则其反向边为 i^1 。这个特性在网络流中应用起来非常方便。